Document number: P0961R1
Audience: EWG

Ville Voutilainen
2018-03-14

Relaxing the structured bindings customization point finding rules

Abstract

This paper proposes relaxing the rules for finding a customization point for structured bindings; that is, just because a a member get() that is not a template is found, that should not disable an ADL customization point. This allows writing ADL get() for types that happen to have a get() member function in their class hierarchy. Examples of such types are smart pointers, future-like types, and user wrappers that happen to inherit from such types.

The proposal in this paper is that, if a member get() is found, it must be a member template to be used with a structured binding. If a get() that is not a templateis found, that member is not used, and the lookup continues to try and find an ADL get<>().

Rationale

It would seem perfectly reasonable to be able to adapt types that have a non-template member get() to be used with structured bindings. Currently, structured bindings will try to use a member get() even when it can never be valid, because it's not a get<>().

Rumination

So, first of all, are the current rules really wrong? It certainly seems reasonable to allow using a member get<>(). I'm not suggesting disallowing that. However, as with the range-for customization point lookup, we seem to lose useful functionality when we have the rules commit to a member as soon as it remotely smells like it may have been intended to be structured bindings customization point.

However, that rationale relies on the assumption that the user somehow intended to opt in to structured bindings, failed to do so correctly, and is now surprised that some namespace-scope get<> was chosen. The problem here is that if the user instead intended to write such a namespace-scope get<>(), and can't change the presence of an 'get' in his class hierarchy, that's just impossible to do.

While it was a nice idea to avoid funny results in the examples discussed during C+17 standardization, I think it's an unfortunate consequence that we now prevent perfectly valid use cases written by programmers who know what they are doing. We shouldn't base our rules on overt worrying about things that could go wrong, but rather allow useful programming techniques to be applied, and rely on tools to hold the hand of a programmer. That is, we used to have a principle of trusting the programmer rather than molly-coddling him.

Is this a significant problem? Probably not, not something that a whole lot of programmers run into. However, the problem seems easy to fix, with not much downside. We have more cases where customization point lookup rules are draconian and prevent reasonable code from working; I have written another paper that deals with a

customization point lookup problem with a range-for loop. These problems can be worked around with adapter types, but I'd rather have less draconian rules.

An example of code that doesn't work

```
#include <memory>
#include <tuple>
#include <string>

struct X : private std::shared_ptr<int>
{
    std::string fun_payload;
};

template<int N> std::string& get(X& x) {if constexpr(N==0) return x.fun_payload;}

namespace std {
    template<> class tuple_size<X> : public std::integral_constant<int, 1> {};
    template<> class tuple_element<0, X> {public: using type = std::string;};
}

int main()
{
    X x;
    auto& [y] = x;
}
```

I find it very unreasonable that that code is ill-formed. I could use some wrapper as a work-around, but in public inheritance cases, I would lose the standard conversion to a base. In addition, that requires users to use the wrapper in part of their client code, but the bare non-wrapped type in other parts.

Side question: if I change the structured binding declaration to auto [y] = x;, gcc and clang complain that they can't bind the (xvalue) argument to the lvalue-reference parameter. I can add an overload for const (but I don't want const here, I want a modifiable object), and I can add an overload for an rvalue (but this seems odd, because I don't want to move). I don't understand why the argument to get<> becomes an xvalue when all I seemed to use was an lvalue? This may work fine for using structured bindings with function return values, but is extremely surprising for binding a named variable. The relevant wording is in [dcl.struct.bind]/3:

> In either case, e is an lvalue if the type of the entity e is an lvalue reference and an xvalue otherwise.

I grok that what is passed to get<> is a copy of x. What I find user-hostile is that defining a get<> that takes a X& and a get<> that takes a const X& is not sufficient.

Wording

The reported issue is intended as a defect report with the proposed resolution as follows. The effect of the wording changes should be applied in implementations of all previous versions of C++ where they apply.

In [dcl.struct.bind]/3, edit as follows:

> Otherwise, if the qualified-id std::tuple_size<E> names a complete type, the expression std::tuple_size<E>::value shall be a well-formed integral constant expression and the number of elements in the identifier-list shall be equal to the value of that expression. The unqualified-id get is looked up in the scope of E by class member access lookup (6.4.5), and if that finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the

initializer is e.get<i>(). Otherwise, the initializer is get<i>(e), where get is looked up in the associated namespaces (6.4.2). In either case, get<i> is interpreted as a template-id.